microPAM_V3 workbench user guide

The workbench is the desktop program of the microPAM_V3 recorder. It uploads firmware, configures the recorder, controls it over USB, and reads, plots and converts its recordings. This guide describes the four tabs of the workbench and, in particular, the different ways in which the recorder can be configured and the recording schedules that the configuration allows. Installation is described in SETUP.md; the recorder hardware, the serial menu and the file format are described in README.md.

1. Starting the workbench

System	Start
Windows	double-click `workbench.cmd` in the package folder
macOS	in Terminal, in the package folder: `Python/.venv/bin/python Python/workbench.py`

The window carries a connection bar at the top and four tabs below it: Firmware, Configuration, Device and Data. The workbench remembers the last serial port, the board's USB serial number, the last firmware file, the last configuration file and the UTC choice of the clock in `Python/workbench_settings.json`.

1.1 The connection bar

Element	Function
Port	the serial ports of the computer; the recorder is `COMn` on Windows and `/dev/cu.usbmodemNNNN` on macOS
↺	refresh the port list, e.g. after plugging in the recorder
Connect / Disconnect	open or close the serial connection
UID	the unique identifier of the board, read at connection; it is also the first part of every file name
Board clock	the board time, as last read
Text size A− / A+	decrease or increase the text size of the whole window (8 to 20 pt, default 11 pt; also Ctrl − / Ctrl +, and Ctrl 0 for the default); the size is kept for the next start, and the window grows to fit, up to the screen size

At connection the workbench asks the board for its UID. After a reboot or an upload it reconnects by itself for up to 10 s, finding the board by its USB serial number, so that a changed port name does not matter. Only one program can hold the serial port, and a serial monitor in another program must therefore be closed before one connects.

2. Where the configuration lives

Before the configuration modes are described, one should know the three places in which a parameter can be held, and in which order they take effect:

Level	Where	When it takes effect
1. Compile-time defaults	`config.h`, built into the firmware	always; overridden by level 2

Level	Where	When it takes effect
2. Configuration file	`/config.txt` on the recorder's microSD card	read once at every boot
3. Running values	the memory of the running recorder, changed over the serial port	at once, until the next boot

Levels 2 and 3 are linked in both directions. The recorder reads `/config.txt` at boot, and it rewrites the file with its running values whenever it stops recording, whenever it goes into hibernation, and on the serial command `w`. Running values that were changed over the serial port are consequently kept if the recorder is stopped, hibernates or receives `w`, but are lost if it is reset or rebooted before. It follows as well that a `config.txt` on the card always shows the values of the last stop, which need not be those that were written onto the card from a PC.

The recording mode `!m` (raw WAV or compressed `.bin`) is the exception: it is fixed when the firmware is built. The recorder writes it into `config.txt` for information, but ignores it when reading the file. The recording mode is changed by uploading the other firmware file (Section 4.1).

3. The configuration parameters

The Configuration tab shows all parameters of `config.txt`, each with its key, its unit and the comment the recorder writes into the file.

Key	Parameter	Unit	Notes
`!a`	file length	s	a new file is started at every multiple of `a` seconds of the board clock
`!o`	on time per repetition period	min	used only if `r` > `o`
`!r`	repetition period	min	`r` ≤ `o` (e.g. 0) means continuous recording
`!f`	sampling frequency	kHz	capped at 192 kHz on the RP2040 and 384 kHz on the RP2350 (192 kHz with four channels)
`!g`	analog gain	dB	0 to 42 dB
`!m`	recording mode		0 raw WAV, 1 compressed; report only (Section 2)
`!s`	ISRC, source		name and calibration, `<name>-<C0>`, e.g. `LC17-38.7` (see README.md, Calibration)
`!c`	ICMS, commissioning organization		
`!n`	IART, operator		

Key	Parameter	Unit	Notes
`!p`	IPRD, project		the serial menu calls this key `k`
`!e`	ISBJ, area		
`!l`	INAM, location identifier		
`!1 !2`	first recording window	h	hours of the board clock
`!3 !4`	second recording window	h	hours of the board clock
`!x`	delayed start time		`yyyy-mm-dd_hh:mm:ss`, board time

The text parameters `s c n p e l x` must be single words of at most 39 characters, as the recorder reads only the first word of a value. They are written into the header of every recording. The workbench checks all fields before it saves or writes them and names any field that the recorder would not accept as intended.

4. Configuration modes

The workbench offers four ways of configuring the recorder. They differ in whether the recorder must be connected, and in when the new values take effect.

Mode	Recorder connected	Takes effect	Typical use
A. Firmware file	yes, by USB	after the upload	choice of recording mode (WAV or compressed)
B. Local file on the SD card	no; the card is in a PC card reader	at the next boot	preparing cards for a deployment, several recorders
C. Board over the serial port	yes, by USB	at once; kept with `w`	adjusting one recorder on the bench
D. Serial monitor commands	yes, by USB	at once	single parameters, checks, scripting by hand

A fifth way, building firmware with other compile-time defaults (developer mode, Section 8), is only needed for settings that are not in `config.txt`, such as the number of channels.

4.1 Mode A: choice of the firmware file

The recording mode is part of the firmware. The package contains one firmware file per board and mode, in the folder `firmware/`:

File ending	Recording	Size of the recordings
`_bin.uf2`	compressed raw samples, `.bin` files	about half of WAV, lossless
`_wav.uf2`	raw 32-bit samples, `.wav` files	full size

Compressed recording is the default and is recommended for deployments, because it stores the same samples in about half the space; the Data tab converts `.bin` files to WAV when a WAV file is needed. Raw WAV recording may be preferred when the files must be read by other software directly on recovery.

To change the mode, one uploads the other file on the Firmware tab (Section 5). The other parameters are not affected, since they are kept in `config.txt` on the card. After the upload, `!m` on the Configuration tab shows the new mode once the configuration has been read from the board.

4.2 Mode B: configuration file for an SD card

In this mode the configuration is prepared on the PC and written to the card in a card reader, without the recorder. It is the mode of choice when several cards are prepared for a deployment, or when the recorder is not at hand.

1. On the Configuration tab, fill the fields. "Defaults from config.h" fills them with the compile-time defaults of the firmware, and "Load…" reads an existing `config.txt`, for example one from a card of an earlier deployment or the example in `data/config.txt`.
2. Edit the fields.
3. Insert the recorder's microSD card in the card reader of the PC.
4. Press "Save as…" and save the file as `config.txt` in the root folder of the card (e.g. `E:\config.txt` on Windows, `/Volumes/<card name>/config.txt` on macOS).
5. Eject the card, insert it in the recorder, and power or reset the recorder. The values are read at boot.

"Save" writes to the file last loaded or saved, whose path is shown below the buttons, and the workbench reloads that file at its next start. The file is written in exactly the layout that the recorder itself writes, so that a file saved by the workbench and one written by the recorder are identical for the same values.

Two points should be kept in mind. First, the key `!m` in the file has no effect on the recording mode (Section 2). Second, the recorder rewrites the file with its running values at every stop, so that a card taken from a recorder carries the values it last used.

4.3 Mode C: configuration of the board over the serial port

In this mode the running recorder is read and changed directly. It requires the recorder to be connected by USB (connection bar, Connect).

1. Press "Read from board". The workbench queries every parameter, and the UID, and fills the fields with the running values.
2. Edit the fields.
3. Press "Write to board". Every parameter except `!m` is sent to the recorder, which applies it at once. With the option "store config.txt on the board's SD card (w)" selected (the default), the recorder then also writes `/config.txt`, so that the new values survive a reboot.

The parameters take effect at once, with the exception of the start time `!x`, which is evaluated only at boot (Section 6.4). A changed sampling frequency or gain applies immediately to the running acquisition. Without the store option, the new values are kept in the recorder's memory only, and they reach the card at the next stop or hibernation; a reboot by the Reboot button (Section 6) before that discards them.

Modes B and C can be combined: "Read from board" followed by "Save as…" keeps a copy of a recorder's configuration on the PC, and "Load…" followed by "Write to board" transfers a prepared configuration to a recorder.

4.4 Mode D: commands in the serial monitor

The Device tab has a serial monitor with an input line, into which the commands of the recorder's serial menu can be typed. `?k` prints parameter `k`, and `!k value` sets it; for example `!a 300` sets 5-minute files, and `?a` prints the file length. Two keys differ from those of `config.txt`: the project IPRD is `k` in the menu (`p` in the file), and the sampling frequency is in Hz in the menu (kHz in the file), so that `!f 96000` sets 96 kHz. The full command list is in README.md (Serial menu). Values set in this way become permanent only with `w`, by a stop, or by a hibernation, as in Mode C.

5. Firmware tab

The Firmware tab has two modes, selected at its top.

User mode (the default) uploads a ready-built firmware file:

1. Select the serial port of the recorder in the connection bar.
2. Press "Browse…" and choose a `.uf2` file; the file dialog opens in the package folder `firmware/`. Below the file name the workbench shows the chip the file was built for, its size and its date.
3. Press Upload.

The workbench releases the serial port, resets the recorder into its boot loader through that port, copies the file onto the boot drive that then appears, and reconnects once the recorder has restarted, switching to the Device tab to show its boot messages. A file built for the other chip (RP2040 or RP2350) is refused before anything is written. A recorder that does not respond on its serial port is put into the boot loader by hand (hold BOOT, press RESET, release BOOT); the upload then copies the file directly.

Developer mode builds the firmware from the source and needs Arduino (SETUP.md, Section 5); it is available on Windows only. It is described in Section 8.

6. Device tab

The Device tab controls the running recorder and shows its serial output.

Control	Command sent	Function
▸ Start (s)	`s`	start recording
■ Stop (e)	`e`	stop recording; the recorder writes `/config.txt`
Print (p)	`p`	print the system and acquisition parameters
Store config (w)	`w`	write `/config.txt` with the running values now
Reboot	`x0`	restart the recorder without rewriting `/config.txt`
Hibernate n hours	`x n`	sleep until the start of the n-th next full hour, then reboot and record
Hibernate n minutes	`y n`	sleep until the start of the n-th next full minute, then reboot and record

Control	Command sent	Function
Read board clock	`?d`	show the board time in the connection bar
Set board clock to PC time	`!d …`	set the board clock to the PC clock, as local time or, with UTC selected, as UTC
Set to	`!d …`	set the board clock to the time typed in, `yyyy-mm-dd hh:mm:ss`

The serial monitor shows everything the recorder prints, including its boot log, and its input line sends any menu command (Section 4.4). Commands that restart the recorder (`x0`, `y0`, `b`) are recognized, and the workbench reconnects after them.

6.1 The board clock

The board clock is kept by the real-time clock of the recorder and carries no time zone. It provides the time stamps of the recordings, the file and folder names, and the time base of all schedules of Section 7. If it is set to UTC (option UTC), the time stamps are true Unix times and the recording hours are UTC hours; if it is set to local time, the stamps and the recording hours are local time. For deployments, one may prefer UTC, as it is free of daylight-saving changes. The clock should be checked, and set if necessary, before every deployment.

7. Recording schedules

The parameters `a`, `o`, `r`, `!1` to `!4` and `!x` together define when the recorder records. All times refer to the board clock (Section 6.1). The recorder checks the schedule each time it closes a file, i.e. at every multiple of `a` seconds, and goes into hibernation when the schedule says that it should not record. In hibernation the recorder draws little current, and it is woken by its real-time clock, reboots and continues recording without waiting for USB.

7.1 Continuous recording

With $r \le o$ (for example `!o 1` and `!r 0`) and recording windows `0 12 12 24`, the recorder records without interruption, in files of `a` seconds. This is the default of the package. The first file after a start is shorter than `a`, because files are closed at multiples of `a` seconds of the clock, e.g. at full minutes for `!a 60`.

7.2 Duty cycle

With $r > o$, the recorder records for `o` minutes at the start of every period of `r` minutes and hibernates for the remainder. The periods are aligned to multiples of `r` minutes of the board clock, so that `!o 5` and `!r 30` record from hh:00 to hh:05 and from hh:30 to hh:35. As the schedule is checked only when a file is closed, the on time is effectively rounded up to whole files, and `a` should divide the on time. With `!a 60` and `!o 5`, five files of 1 minute are written per period; with `!a 600` and `!o 5`, by contrast, one file of 10 minutes is written, and the on time is twice as long as intended. In addition, `r` should divide 1440 (minutes per day), so that the periods fall on the same times every day.

As a check of the storage needed: raw WAV recording at 192 kHz with one channel writes 4 bytes per sample, i.e. 0.77 MB/s or 2.8 GB per hour. Compressed recording needs about half of that, some 1.4 GB per hour, whereby the exact figure depends on the signal, as the compression

is lossless. A duty cycle of 5 minutes in 30 records 4 hours per day and then fills about 6 GB per day, against about 33 GB per day for continuous compressed recording.

7.3 Recording hours

The four values `!1` to `!4` define two daily recording windows, in whole hours of the board clock. An hour h is recorded if `!1` ≤ h < `!2` or `!3` ≤ h < `!4`, i.e. each window starts at its first hour and ends at its second, as clock times are read; outside the windows the recorder hibernates until the next window opens. The values must be in ascending order, from 0 to 24, and a window whose two hours are equal is empty.

`!1 !2 !3 !4`	Recorded
`0 12 12 24`	always (the default)
`6 18 18 18`	06:00 to 18:00
`0 6 20 24`	00:00 to 06:00 and 20:00 to 24:00, i.e. the night
`6 9 18 21`	06:00 to 09:00 and 18:00 to 21:00
`0 6 18 24`	18:00 to 06:00 of the next morning, every night

A window cannot run across midnight, because the hours are compared within one day. A recording over night, from 18:00 to 06:00 of the next morning, is therefore written as two windows: the morning part first (`!1 !2` = `0 6`) and the evening part second (`!3 !4` = `18 24`). The two parts join at midnight, so that the recorder records without a break from 18:00 to 06:00 and hibernates from 06:00 to 18:00. Writing the evening window first, as `18 24 0 6`, does not work: the values are then not in ascending order, and the recorder would record a single file at 18:00 and then hibernate until 18:00 of the next day, because every hour from 6 onwards counts as past the end of the day's windows.

Such an overnight schedule may be started in two ways. If the recorder is simply started during the day, it records one file, finds itself outside its windows and hibernates until 18:00. If, in addition, the start time `!x` is set to 18:00 of the first evening (Section 7.4), the recorder hibernates from its start until then, and this stray daytime file is avoided.

Firmware before version 3.4.2 recorded the end hours in full, so that `0 6 20 24` recorded until 06:59 there; a `config.txt` prepared for older firmware therefore records one hour less per window with 3.4.2 and later.

Recording hours and duty cycle can be combined: within the windows the duty cycle of Section 7.2 applies. When the recorder is started outside its windows, it first records one file and then hibernates, since the schedule is checked at the end of each file.

7.4 Delayed start

The start time `!x` (`yyyy-mm-dd_hh:mm:ss`, board time) delays the start of the recording. At boot the recorder compares it with its clock; if the start time lies in the future, and less than a year ahead, the recorder hibernates until then. A start time in the past, such as the default `2000-01-01_00:00:00`, starts the recording immediately.

Because the start time is evaluated only at boot, a delayed start is prepared in one of two ways:

- Mode B: set `!x` in the `config.txt` of the card, insert the card and power the recorder.

- Mode C: set `!x` and press “Write to board” with the store option selected; then press Reboot on the Device tab.

The real-time clock wakes the recorder on a match of hour and minute. For a start more than a day ahead the recorder therefore wakes once a day at that time of day, finds the start still in the future and goes back to sleep, until the start date is reached. When the recorder hibernates, for whatever reason, it stores its wake-up time as `!x`, so that after a hibernation `!x` holds the last wake-up time, which lies in the past and is harmless.

7.5 Hibernation by hand

The Hibernate controls of the Device tab put the recorder to sleep until the start of the n-th next full hour or minute. “Hibernate 2 hours” at 10:20, for example, sleeps until 12:00. This is useful to start a recording at a full hour on the bench, or to check a schedule. The command `x0` (Reboot) does not hibernate but restarts the recorder at once.

8. Developer mode (Windows only)

Developer mode on the Firmware tab builds the firmware from the source in the package and uploads it. It needs Arduino IDE 2 with the board package “Raspberry Pi Pico/RP2040/RP2350” by Earle F. Philhower (SETUP.md, Section 5); until that is found, the mode is disabled and the tab names what is missing, and “Check again” re-tests without a restart.

Control	Function
Target	RP2040 Feather Adalogger or RP2350 Feather HSTX
PROC_MODE (config.h)	0 raw WAV or 1 compressed; selecting a value edits `config.h` at once, and a rebuild applies it
Build	compile only
Upload	upload the last build
Build + Upload	compile and upload
Clean	remove the build output
Stop	abort a running build or upload

Developer mode is needed only for compile-time settings that `config.txt` does not carry, such as the channel selection `NCHAN` or the preamplifier type. These are edited in `config.h` and `src/global.h` with a text editor; the compile-time defaults of all `config.txt` parameters are also in `config.h`. The recording mode alone does not require developer mode, because both modes are available as firmware files.

9. Data tab

The Data tab browses recordings on a local disk or on a microSD card in a card reader.

Control	Function
⬆ Up, ⟳ Refresh, 📁 Open folder…	navigate; double-click opens a folder
Header	the header of the selected file: UID, settings, metadata, calibration (double-click on a file does the same)

Control	Function
View data	time series and spectrogram of the selected file; the colour range of the spectrogram is adjusted in 5 dB steps or reset
Spectrum	Welch spectrum of the selected file
To WAV…	convert the selected `.bin` files, and all `.bin` files in selected folders, into a chosen folder

The right mouse button on a file or folder opens the same functions as a menu, together with "Open in Explorer" (Windows) or "Open in Finder" (macOS). If the source string `!s` carries a calibration (Section 3), the time series is shown in Pa and the spectra in dB re 1 µPa²/Hz; otherwise the plots are in counts. The conversion to WAV keeps the folder structure below a selected folder, and records that were missed during acquisition are filled with zeros, so that the WAV file keeps the timing of the samples.

10. Checklist for a deployment

1. Charge or connect the battery, insert a formatted microSD card, connect USB, Connect.
2. Firmware tab: upload the firmware file of the wanted recording mode, if it differs from the installed one.
3. Device tab: set the board clock (preferably to UTC) and read it back.
4. Configuration tab: "Read from board", set file length, sampling frequency, gain, metadata (location `!l`, area `!e`, project `!p`) and the schedule (Section 7), then "Write to board" with the store option selected.
5. Device tab: Print (p) and check the parameters in the monitor; for a delayed start, press Reboot and check that the boot log reports the hibernation.
6. Disconnect and deploy. After recovery, check the files on the Data tab, in particular `missed: acq` in the ICMT entry of the header (INFO chunks), which counts records lost at acquisition.